NESNE YÖNELİMLİ PROGRAMLAMA-1

Python'da Temel Veri Yapıları

Liste, demet, sözlük ve değişken kapsamı

`list [ ]` `tuple ( )` `dict { }` `scope`

```
hafta1.py
ders = {
    "hafta": 1,
    "konular": ["liste",
        "demet", "sözlük",
        "kapsam"],
}

print("Hazır mıyız?")
```

Erhan AKAGÜNDÜZ

BU HAFTA

Dersin sonunda neler yapabileceksiniz?

Liste

Oluşturmak, indekslemek, dilimlemek, eleman eklemek ve silmek

Demet

Değiştirilemez veriyi saklamak, açmak ve fonksiyondan döndürmek

Sözlük

Anahtar–değer çiftleriyle veriyi isimlendirerek yönetmek

Kapsam

Bir değişkenin nerede görünür olduğunu LEGB kuralıyla açıklamak

Neden OOP dersinde? 3. haftadan itibaren yazacağımız her sınıfın içindeki veriler bu yapılarla tutulacak. Kapsamı bilmeden **self** parametresini anlamak zordur.

MOTİVASYON

30 öğrencinin notunu nasıl saklarız?

⚠ Yol 1: Tek tek değişken

```
not1 = 85
not2 = 92
not3 = 78
# ... not30 = ?

ort = (not1 + not2 + not3 ...) / 30
```

30 isim, 30 satır. Öğrenci sayısı değişince kodu baştan yazmak gerekir.

✓ Yol 2: Bir veri yapısı

```
notlar = [85, 92, 78, ...]



ort = sum(notlar) / len(notlar)
```

Tek isim, tek satır. 3 öğrenci de olsa 3000 öğrenci de olsa aynı kod çalışır.

Veri yapısı = birçok değeri tek bir isim altında düzenli tutmanın yolu.

GENEL BAKIŞ

Üç yapı, üç farklı karakter

[ ]

Liste • list

Alışveriş listesi gibi: **sıralı**, eleman eklenip çıkarılabilir.

```
["süt", "ekmek"]
```

( )

Demet • tuple

Doğum tarihi gibi: **sıralı** ama oluşturulduktan sonra **değişmez**.

```
(15, 6, 2005)
```

{ }

Sözlük • dict

Telefon rehberi gibi: değere **anahtar** (isim) ile ulaşılır.

```
{"ad": "Can"}
```

☆ **Sonra da:** Bu yapıları tutan değişkenler programın neresinden görülebilir? → **Değişken kapsamı**

BÖLÜM 1

01

Liste [ ]

Sıralı, değiştirilebilir ve Python'da en çok kullanılan koleksiyon.

indeksleme | dilimleme | ekle / sil | sıralama | comprehension | referans tuzağı

LİSTE · KAVRAM

Liste, numaralı vagonlardan oluşan bir trendir



Her vagonun bir **sıra numarası (indeks)** var ve numaralar **0'dan** başlar. Bir vagonda başka bir liste bile taşınabilir.

Sıralı

Elemanlar eklendiği sırayı korur.

Değiştirilebilir

Oluşturduktan sonra eklenir, silinir (mutable).

Tekrara izin verir

Aynı değer birden çok kez yer alabilir.

Karışık türler

Farklı türler aynı listede durabilir.

Köşeli parantez ve virgül: hepsi bu

liste_olustur.py

```
notlar = [85, 92, 78, 64, 90]
karisik = ["Ali", 21, 3.75, True]
bos = []

print(notlar)
print(karisik)
print(len(notlar), type(notlar))
```

▷ ÇIKTI

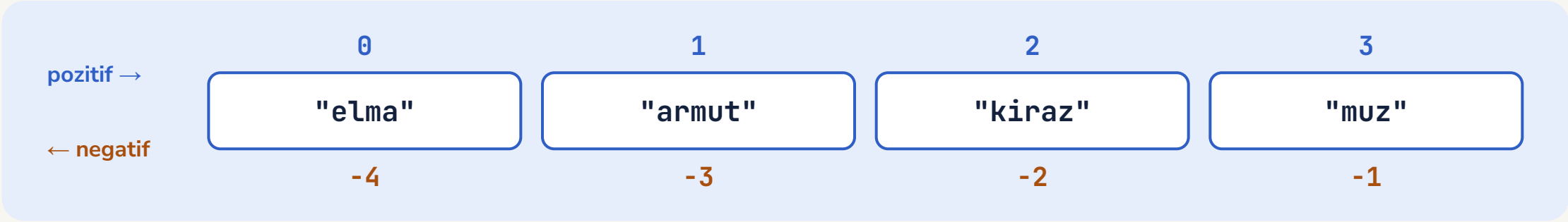
```
[85, 92, 78, 64, 90]
['Ali', 21, 3.75, True]
5 <class 'list'>
```

len() eleman sayısını, **type()** nesnenin türünü verir. Boş liste **[]** veya **list()** ile oluşturulur.

</> Python ekrana yazarken metinleri tek tırnakla gösterir: **'Ali'**. İkisi de aynı şeydir.

LİSTE · İNDEKSLEME

Başştan say: 0, 1, 2... Sondan say: −1, −2...



indeks.py

```
meyveler = ["elma", "armut", "kiraz", "muz"]
print(meyveler[0])
print(meyveler[2])
print(meyveler[-1])
print(meyveler[-2])
```

▷ ÇIKTI

```
elma
kiraz
muz
kiraz
```

⚠ **meyveler[4]** → IndexError. Son indeks her zaman **len − 1**'dir.

liste[başlangıç : bitiş : adım]



sayilar[1:4]

1 **dahil**, 4 **hariç** → indeks 1, 2, 3 seçilir. Kesikli kutu (4) alınmaz.

dilim.py

```
sayilar = [10, 20, 30, 40, 50, 60]
print(sayilar[1:4])
print(sayilar[:3])   # baştan
print(sayilar[3:])   # sona kadar
print(sayilar[::2])  # ikişer atla
print(sayilar[::-1]) # ters çevir
```

▷ ÇIKTI

```
[20, 30, 40]
[10, 20, 30]
[40, 50, 60]
[10, 30, 50]
[60, 50, 40, 30, 20, 10]
```

append, insert, extend: üç farklı ekleme

ekleme.py

```
sepet = ["süt", "ekmek"]
sepet.append("yumurta")
print(sepet)
sepet.insert(1, "peynir")
print(sepet)
sepet.extend(["zeytin", "çay"])
print(sepet)
```

▷ ÇIKTI

```
['süt', 'ekmek', 'yumurta']
['süt', 'peynir', 'ekmek', 'yumurta']
['süt', 'peynir', 'ekmek', 'yumurta', 'zeytin', 'çay']
```

`append(x)`

Tek bir elemanı listenin **sonuna** ekler.

`insert(i, x)`

Elemanı **i** konumuna koyar; sonrakiler bir sağa kayar.

`extend(liste)`

Başka bir listenin **tüm elemanlarını** tek tek ekler.

⚠ **append(["a","b"])** listeyi tek eleman olarak ekler: iç içe liste oluşur.

Değere göre mi, konuma göre mi?

silme.py

```
sepet = ["süt", "peynir", "ekmek",
         "yumurta", "çay"]
sepet.remove("ekmek")
print(sepet)
son = sepet.pop()
print(son, sepet)
del sepet[0]
print(sepet)
```

▷ ÇIKTI

```
['süt', 'peynir', 'yumurta', 'çay']
çay ['süt', 'peynir', 'yumurta']
['peynir', 'yumurta']
```

Yöntem	Ne yapar?
remove(x)	İlk x değerini siler
pop()	Sondakini siler ve döndürür
pop(i)	i konumundakini siler, döndürür
del liste[i]	i konumundakini siler
clear()	Listeyi tamamen boşaltır

⚠ Listede olmayan bir değeri **remove** etmek ValueError verir. Önce **in** ile kontrol edin.

sorted() yeni liste verir, sort() yerinde değiştirir

siralama.py

```
notlar = [78, 92, 64, 85, 90]
print(sorted(notlar))
print(notlar)  # değişmedi!

notlar.sort(reverse=True)
print(notlar)

print(max(notlar), min(notlar), sum(notlar))
print(92 in notlar, notlar.index(85))
```

▷ ÇIKTI

```
[64, 78, 85, 90, 92]
[78, 92, 64, 85, 90]
[92, 90, 85, 78, 64]
92 64 409
True 2
```

in "bu eleman listede var mı?" sorusunu True/False ile yanıtlar. **index()** ilk bulunduğu konumu verir.

LİSTE · BAŞVURU KARTI

En çok kullanılan liste işlemleri

İşlem	Örnek	Sonuç (l = [3, 1, 2])	Listeyi değiştirir mi?
Uzunluk	len(l)	3	Hayır
Sona ekle	l.append(4)	[3, 1, 2, 4]	Evet
Araya ekle	l.insert(0, 9)	[9, 3, 1, 2]	Evet
Değerle sil	l.remove(1)	[3, 2]	Evet
Sıralı kopya	sorted(l)	[1, 2, 3]	Hayır
Yerinde sırala	l.sort()	l → [1, 2, 3]	Evet
Ters çevir	l.reverse()	l → [2, 1, 3]	Evet
Say	l.count(1)	1	Hayır
Birleştir	l + [5, 6]	[3, 1, 2, 5, 6]	Hayır

Döngüyü tek satırlık listeye çevirmek

```
[ ifade for eleman in kaynak if koşul ]
```

ifade: listeye ne girecek? **eleman:** döngü değişkeni **kaynak:** nereden geliyor? **koşul:** isteğe bağlı süzgeç

comprehension.py

```python
kareler = [x ** 2 for x in range(1, 6)]
print(kareler)
ciftler = [x for x in range(10) if x % 2 == 0]
print(ciftler)
isimler = ["ayşe", "mehmet", "zeynep"]
print([i.capitalize() for i in isimler])
```

▷ ÇIKTI

```
[1, 4, 9, 16, 25]
[0, 2, 4, 6, 8]
['Ayşe', 'Mehmet', 'Zeynep']
```

Okunması zorlaşıyorsa klasik **for** döngüsüne dönün. Kısa kod her zaman iyi kod değildir.

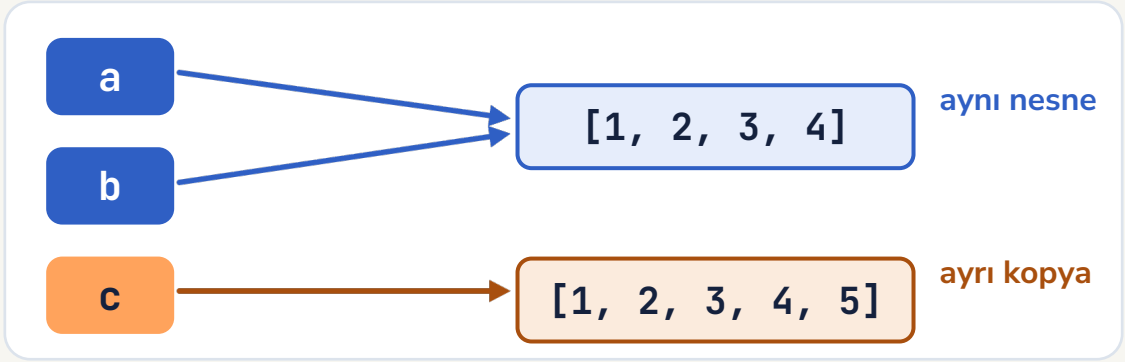
b = a kopyalamaz, aynı listeye ikinci bir isim verir

referans.py

```
a = [1, 2, 3]
b = a
b.append(4)
print("a:", a)
print("b:", b)
print(a is b)
c = a.copy()
c.append(5)
print("a:", a, "c:", c)
```

▷ ÇIKTI

```
a: [1, 2, 3, 4]
b: [1, 2, 3, 4]
True
a: [1, 2, 3, 4] c: [1, 2, 3, 4, 5]
```



BÖLÜM 2

02

Demet ( )

Sıralı ama kilitli: oluşturulduktan sonra değiştirilemeyen koleksiyon.

değişmezlik | açma (unpacking) | çoklu dönüş | liste mi demet mi?

Demet, mühürlenmiş bir listedir



Ankara'nın koordinatları değişmez; bu yüzden demet olarak saklamak mantıklı. Elemanlar **okunur** ama **değiştirilemez**.

Tipik kullanımlar

- koordinat (x, y)
- RGB renk (255, 0, 0)
- tarih (gün, ay, yıl)
- haftanın günleri

demet.py

```
konum = (39.92, 32.85)
renk = ("kırmızı", "yeşil", "mavi")
tek = (5,)    # virgül şart!
yanlis = (5)
print(konum[0], renk[-1])
print(type(tek), type(yanlis))
```

▷ ÇIKTI

```
39.92 mavi
<class 'tuple'> <class 'int'>
```

⚠ Demeti parantez değil **virgül** oluşturur. (5) sadece parantez içinde bir sayıdır.

Değiştirmeye çalışırsak: TypeError

degismez.py

```
gunler = ("Pzt", "Sal", "Çar")
try:
    gunler[0] = "Paz"
except TypeError as e:
    print("Hata:", e)

# Demetin sadece iki metodu var
print(gunler.count("Sal"), gunler.index("Çar"))
```

▷ ÇIKTI

```
Hata: 'tuple' object does not support item assignment
1 2
```

Neden değişmezlik iyi bir şey?

- **Güvenlik:** Veri yanlışlıkla bozulamaz.
- **Hız:** Listeden biraz daha az bellek harcar.
- **Sözlük anahtarı** olabilir (birazdan göreceğiz).
- **Niyet bildirir:** “Bu veri sabit” der.

try / except hatayı yakalayıp programın çökmesini önler. 13. haftada ayrıntılı işleyeceğiz.

Paketi aç, her parçayı bir değişkene koy



unpacking.py

```python
ogrenci = ("Elif", 20, "Bilgisayar")
ad, yas, bolum = ogrenci
print(ad, "-", yas, "-", bolum)
x, y = 5, 10
x, y = y, x     # yer değiştir
print("x =", x, "y =", y)

ilk, *orta, son = [1, 2, 3, 4, 5]
print(ilk, orta, son)
```

▷ ÇIKTI

```
Elif - 20 - Bilgisayar
x = 10 y = 5
1 [2, 3, 4] 5
```

Değişken sayısı eleman sayısına **eşit** olmalı. ***orta** "geri kalan her şeyi" bir listeye toplar.

Bir fonksiyon birden fazla değer döndürebilir mi?

istatistik.py

```python
def istatistik(sayilar):
    en_az = min(sayilar)
    en_cok = max(sayilar)
    ort = sum(sayilar) / len(sayilar)
    return en_az, en_cok, ort  # demet!

sonuc = istatistik([70, 85, 90, 55])
print(sonuc, type(sonuc))

k, b, o = istatistik([70, 85, 90, 55])
print("En düşük:", k, "Ort:", o)
```

▷ ÇIKTI

```
(55, 90, 75.0) <class 'tuple'>
En düşük: 55 Ort: 75.0
```

Perde arkasında ne oluyor?

return a, b, c aslında tek bir demet döndürür: **(a, b, c)**. Çağıran taraf isterse onu bütün olarak saklar, isterse açar.

Liste mi, demet mi?

[ ] Liste - List

- Değiştirilebilir (mutable)
- Çok sayıda metot: append, remove, sort…
- Eleman sayısı zamanla **değişecekse**
- Genelde **aynı türden** elemanlar

```
sepet = ["süt", "ekmek"]
```

( ) Demet - Tuple

- Değiştirilemez (immutable)
- Yalnızca iki metot: count, index
- Veri **sabit** kalacaksa
- Genelde **farklı anlamlı** alanlar

```
ogrenci = ("Elif", 20)
```

Basit kural: değişecekse liste, sabitse demet.

BÖLÜM 3

03

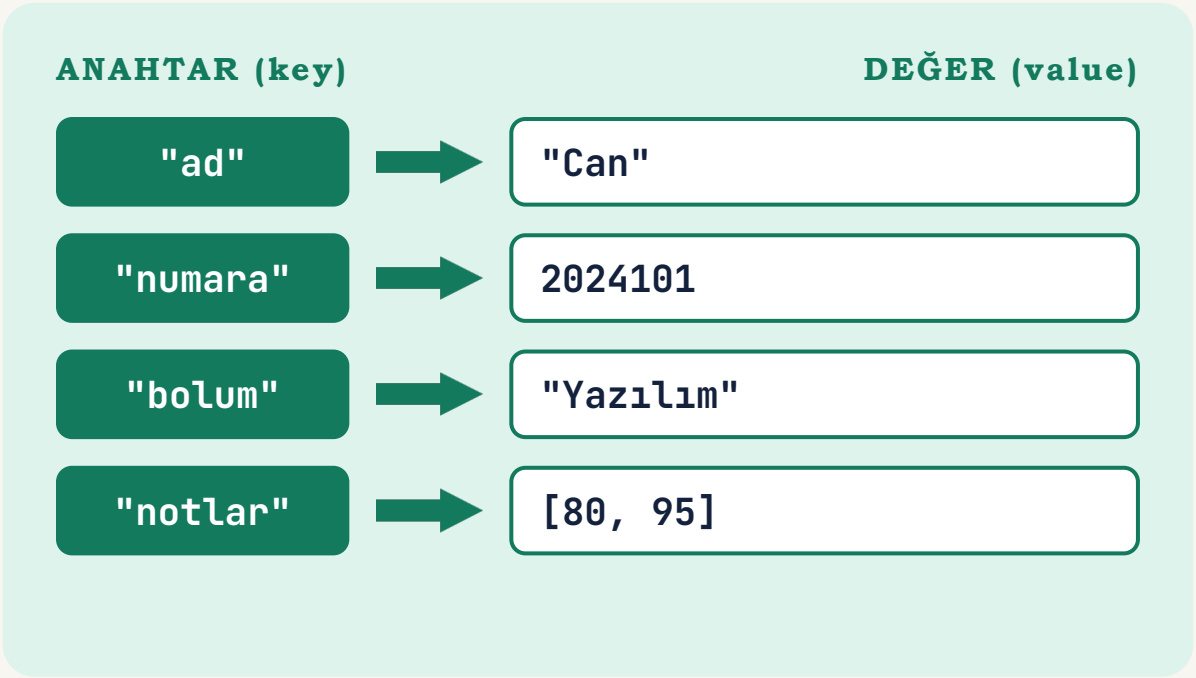
Sözlük { }

Değerlere sıra numarasıyla değil, anlamlı bir isimle (anahtarla) ulaşmak.

anahtar → değer | get() | güncelleme | döngü | iç içe yapılar

SÖZLÜK (dictionary) · KAVRAM

Telefon rehberinde numara değil, isim ararsınız



Listeyle

```
ogrenci[2]
```

“2 neydi?” Okuyan tahmin etmek zorunda.

Sözlükle

```
ogrenci["bolum"]
```

Kod kendini açıklıyor.

- Anahtarlar **benzersizdir**
- Değiştirilebilir (mutable)
- Ekleme sırasını korur (Python 3.7+)

Süslü parantez, iki nokta ve köşeli parantezle erişim

sozluk.py

```
ogrenci = {
    "ad": "Can",
    "numara": 2024101,
    "bolum": "Yazılım",
    "notlar": [80, 95]
}
print(ogrenci["ad"])
print(ogrenci["notlar"][1])
print(len(ogrenci))
```

▷ ÇIKTI

```
Can
95
4
```

ogrenci["notlar"][1] iki adımda okunur: önce sözlükten listeyi al, sonra listenin 1. indeksini.

len() sözlükte **anahtar sayısını** verir. Boş sözlük: **{}** veya **dict()**.

SÖZLÜK · GÜVENLİ ERİŞİM

Anahtar yoksa: [ ] mi, get() mi?

get.py

```python
ogrenci = {"ad": "Can", "bolum": "Yazılım"}
print(ogrenci.get("ad"))
print(ogrenci.get("telefon"))
print(ogrenci.get("telefon", "Yok"))
try:
    print(ogrenci["telefon"])
except KeyError as e:
    print("KeyError:", e)
```

▷ ÇIKTI

```
Can
None
Yok
KeyError: 'telefon'
```

`sozluk["anahtar"]`

Anahtar yoksa **KeyError** fırlatır ve program durur.

`sozluk.get("anahtar")`

Anahtar yoksa **None** döner, hata vermez.

`sozluk.get("anahtar", v)`

Anahtar yoksa senin belirlediğin **varsayılan** değeri döner.

Anahtar varsa günceller, yoksa ekler

guncelle.py

```
urun = {"ad": "Kalem", "fiyat": 15}
urun["fiyat"] = 18    # var → güncelle
urun["stok"] = 120    # yok → ekle
print(urun)
urun.update({"renk": "mavi", "fiyat": 20})
print(urun)
del urun["renk"]
print(urun.pop("stok"), urun)
```

▷ ÇIKTI

```
{'ad': 'Kalem', 'fiyat': 18, 'stok': 120}
{'ad': 'Kalem', 'fiyat': 20, 'stok': 120, 'renk': 'mavi'}
120 {'ad': 'Kalem', 'fiyat': 20}
```

`d[k] = v`

Ekle ya da üzerine yaz.

`d.update({...})`

Birden çok çifti tek seferde ekle/güncelle.

`del d[k]`

Çifti sil.

`d.pop(k)`

Çifti sil ve **değerini döndür**.

keys(), values(), items(): üç farklı bakış

dongu.py

```
baskentler = {
    "Türkiye": "Ankara",
    "Fransa": "Paris",
    "Japonya": "Tokyo",
}
for ulke, sehir in baskentler.items():
    print(f"{ulke:<8} -> {sehir}")

print(list(baskentler.keys()))
print("Fransa" in baskentler)
```

▷ ÇIKTI

```
Türkiye  -> Ankara
Fransa   -> Paris
Japonya  -> Tokyo
['Türkiye', 'Fransa', 'Japonya']
True
```

Metot	Ne verir?
keys()	Anahtarlar
values()	Değerler
items()	(anahtar, değer) demetleri

in sözlükte **anahtarlar** arasında arar, değerlerde değil.

Mini uygulama: kelime sayacı

sayac.py

```
metin = "elma armut elma kiraz armut elma"
sayac = {}
for kelime in metin.split():
    sayac[kelime] = sayac.get(kelime, 0) + 1
print(sayac)
```

▷ ÇIKTI

```
{'elma': 3, 'armut': 2, 'kiraz': 1}
```

get(kelime, 0) kelime ilk kez görülüyorsa 0 verir. Böylece KeyError almadan saymaya başlarız.

Adım adım izleme

#	kelime	sayac (döngü sonrası)
1	elma	{elma: 1}
2	armut	{elma: 1, armut: 1}
3	elma	{elma: 2, armut: 1}
4	kiraz	{elma: 2, armut: 1, kiraz: 1}
5	armut	{elma: 2, armut: 2, kiraz: 1}
6	elma	{elma: 3, armut: 2, kiraz: 1}

Tek satırda sözlük üretmek ve süzmek

```
{ anahtar: değer for eleman in kaynak if koşul }
```

dict_comp.py

```python
kareler = {x: x ** 2 for x in range(1, 6)}
print(kareler)

notlar = {"Ali": 45, "Ece": 82, "Mert": 67}
gecenler = {ad: n for ad, n in notlar.items()
            if n >= 50}
print(gecenler)
```

▷ ÇIKTI

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
{'Ece': 82, 'Mert': 67}
```

Liste comprehension ile aynı mantık; tek fark **süslü parantez** ve **anahtar: değer** çifti.

Sözlüğün içinde sözlük, onun içinde liste

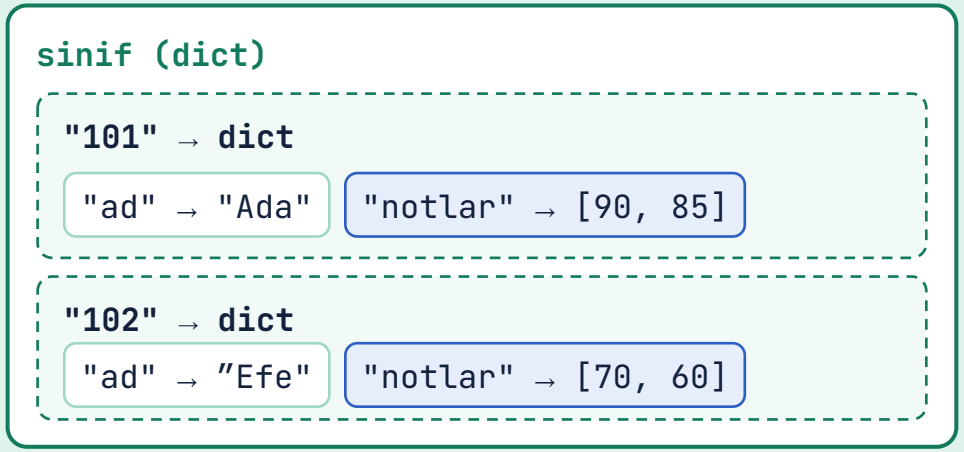
sinif.py

```
sinif = {
    "101": {"ad": "Ada", "notlar": [90, 85]},
    "102": {"ad": "Efe", "notlar": [70, 60]},
}
for no, bilgi in sinif.items():
    n = bilgi["notlar"]
    ort = sum(n) / len(n)
    print(no, bilgi["ad"], ort)
```

▷ ÇIKTI

```
101 Ada 87.5
102 Efe 65.0
```

YAPININ HARİTASI



`sinif["101"]["notlar"][0]` → 90

Anahtar değiştirilemez (hashable) olmalı

anahtar.py

```
d = {}
d[(1, 2)] = "demet olur"
d["ad"] = "metin olur"
print(d)
try:
    d[[1, 2]] = "liste olmaz"
except TypeError as e:
    print("Hata:", e)
```

▷ ÇIKTI

```
{(1, 2): 'demet olur', 'ad': 'metin olur'}
Hata: unhashable type: 'list'
```

Anahtar olabilir

`str` · `int` · `float` · `bool` · `tuple`

Anahtar olamaz

`list` · `dict`

Değişebilen bir şey anahtar olsaydı, değiştiğinde sözlük onu bir daha bulamazdı. **Değerler** ise her şey olabilir.

ARA ÖZET

Üç yapı yan yana

Özellik	Liste	Demet	Sözlük
Yazım	[1, 2, 3]	(1, 2, 3)	{"a": 1, "b": 2}
Erişim	indeks: l[0]	indeks: t[0]	anahtar: d["a"]
Değiştirilebilir mi?	Evet	Hayır	Evet
Sıra korunur mu?	Evet	Evet	Evet (ekleme sırası)
Tekrar olabilir mi?	Evet	Evet	Anahtar: hayır, değer: evet
Sözlük anahtarı olur mu?	Hayır	Evet	Hayır
Tipik örnek	Not listesi	Koordinat, tarih	Öğrenci kaydı

Liste: “aynı şeyden birçok”

Demet: “sabit bir paket”

Sözlük: “isimli alanlar”

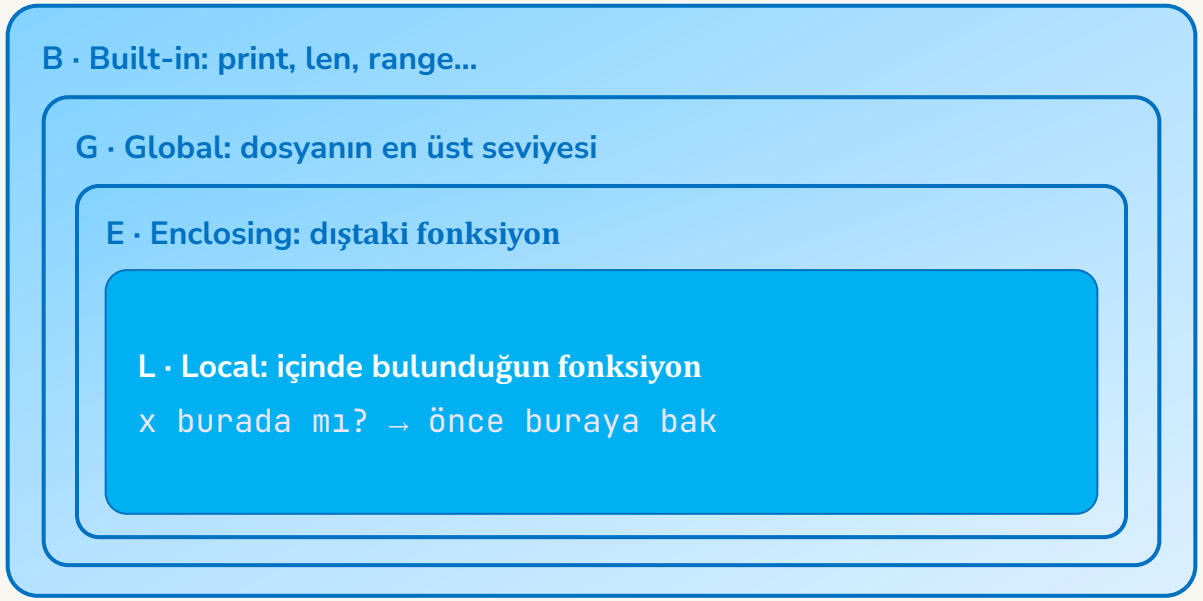
BÖLÜM 4

04 Değişken Kapsamı

Bir değişken programın neresinde “görünür”?
Python bir ismi nerede arar?

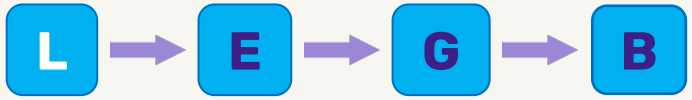
yerel (local) · global · gölgeleme · nonlocal · LEGB kuralı

İç içe odalar: içeriden dışarıyı görürsün, tersi olmaz



Kapsam (scope): bir ismin tanımlı ve erişilebilir olduğu kod bölgesi.

PYTHON BİR İSMİ ŞU SIRAYLA ARAR



İlk bulduğu yerde durur. Hiçbir yerde bulamazsa **NameError** verir.

Odandaki eşya sana ait; salondakini her oda görür. Ama salondan odandaki çekmeceyi açamazsın.

KAPSAM · YEREL (LOCAL)

Fonksiyonda doğan, fonksiyonla ölür

yerel.py

```python
def selamla():
    mesaj = "Merhaba!"    # yerel değişken
    print(mesaj)

selamla()
try:
    print(mesaj)    # dışarıdan erişim
except NameError as e:
    print("NameError:", e)
```

▷ ÇIKTI

```
Merhaba!
NameError: name 'mesaj' is not defined
```

Bir yerel değişkenin hayatı

1. selamla() çağrılır
2. mesaj oluşturulur
3. fonksiyon biter
4. mesaj **yok olur**

Fonksiyon **parametreleri** de yerel değişkendir.

Global değişkeni her fonksiyon okuyabilir

global.py

```
okul = "Python Akademi"   # global

def bilgi():
    print("Okul:", okul)   # okunabilir

bilgi()
print(okul)
```

▷ ÇIKTI

```
Okul: Python Akademi
Python Akademi
```

LEGB ile izleyelim

okul ismi bilgi() içinde aranıyor:

L Local'de var mı? **Hayır**

E Dış fonksiyon var mı? **Yok**

G Global'de var mı? **Evet → bulundu**

⚠ **Okumak** serbest. Ama fonksiyon içinde global değişkene **atama** yapmak bambaşka bir hikâye. →

İçeride atama yaparsan yeni, yerel bir değişken doğar

golge.py

```
x = 10

def f():
    x = 99        # yeni, yerel bir x
    print("içeride:", x)

f()
print("dışarıda:", x)
```

▷ ÇIKTI

```
içeride: 99
dışarıda: 10
```

GLOBAL ALAN

`x = 10`

f() İÇİ

`x = 99`

Dıştaki x'i örter (gölgeler) ama değiştirmez.

Aynı isim, **iki ayrı** değişken. Kafa karıştırır; gerçek kodda farklı isimler tercih edin.

KAPSAM · KLASİK TUZAK

UnboundLocalError: neden artıramıyorum?

tuzak.py

```
sayac = 0
def artir():
    try:
        sayac = sayac + 1
    except UnboundLocalError as e:
        print("Hata:", e)

artir()
```

▷ ÇIKTI (Python 3.11+)

```
Hata: cannot access local variable 'sayac' where it is not associated with a value
```

Ne oldu?

1. Fonksiyonda **sayac = ...** ataması var.
2. Python sayac'ı **yerel** kabul eder.
3. Sağ taraftaki **sayac + 1** okunurken yerel sayac henüz **boş**.

Eski sürümlerde mesaj: *local variable 'sayac' referenced before assignment*. Çözüm sonraki slaytta.

“Bu isim dışarıdaki değişken” demenin yolu

global_kw.py

```
sayac = 0
def artir():
    global sayac
    sayac += 1

artir(); artir(); artir()
print("sayac =", sayac)
```

▷ ÇIKTI

```
sayac = 3
```

⚠ Çalışır ama dikkatli kullanın

- Her fonksiyon değiştirebilirse hata kaynağını bulmak zorlaşır.
- Fonksiyonu test etmek ve yeniden kullanmak güçleşir.

Daha temiz yol: al, işle, döndür

```
def artir(s):
    return s + 1

sayac = artir(sayac)
```

Dış fonksiyonun değişkenini değiştirmek

nonlocal.py

```
def disari():
    sayi = 0          # enclosing
    def iceri():
        nonlocal sayi
        sayi += 1
        return sayi
    iceri(); iceri()
    print("sayi =", sayi)
disari()
```

▷ ÇIKTI

```
sayi = 2
```

Anahtar kelime	Hangi değişkeni bağlar?
(hiçbiri)	Atama → yeni yerel değişken
global	Modülün en üst seviyesindeki
nonlocal	Bir dış fonksiyondaki

nonlocal olmasaydı iceri() içindeki **sayi += 1** yine UnboundLocalError verirdi.

İç fonksiyonun dıştaki değişkeni "hatırlaması" **closure** (kapanış) olarak bilinir.

KAPSAM · HEPSİ BİR ARADA

Aynı isim, dört katman: LEGB iş başında

legb.py

```
x = "global"
def dis():
    x = "enclosing"
    def ic():
        x = "local"
        print("ic:", x)
    ic()
    print("dis:", x)

dis()
print("modül:", x)
print(len("abc"))   # len → built-in
```

▷ ÇIKTI

```
ic: local
dis: enclosing
modül: global
3
```

L ic() kendi x'ini buldu, aramayı bitirdi

E dis() kendi x'ini görür; ic()'ninkini değil

G Modül seviyesi sadece global x'i görür

B len hiçbir yerde tanımlı değil → yerleşik

Listeyi değiştir, dışarısı da değişir

arguman.py

```
def ekle(liste, oge):
    liste.append(oge)
alisveris = ["süt"]
ekle(alisveris, "ekmek")
print(alisveris)

def degistir(sayi):
    sayi += 100

n = 5
degistir(n)
print(n)
```

▷ ÇIKTI

```
['süt', 'ekmek']
5
```

Liste (mutable) → değişti

liste ve alisveris **aynı nesneyi** gösterir. append o nesneyi yerinde değiştirir.

int (immutable) → değişmedi

sayi += 100 **yeni bir sayı** üretir ve yerel sayi'ya bağlar. Dıştaki n etkilenmez.

Mini proje: sınıf not raporu

not_raporu.py

```python
ogrenciler = [
    {"ad": "Ada", "notlar": (90, 85, 95)},
    {"ad": "Efe", "notlar": (60, 45, 70)},
    {"ad": "Nil", "notlar": (75, 80, 70)},
]
def ortalama(notlar):
    return sum(notlar) / len(notlar)

rapor = {}
for o in ogrenciler:
    rapor[o["ad"]] = round(ortalama(o["notlar"]), 1)
print(rapor)
en_iyi = max(rapor, key=rapor.get)
print("En yüksek:", en_iyi, rapor[en_iyi])
```

▷ ÇIKTI

```
{'Ada': 90.0, 'Efe': 58.3, 'Nil': 75.0}
En yüksek: Ada 90.0
```

HANGİ KAVRAM NEREDE?

- **Liste** — ogrenciler: öğrenci kayıtlarının sırası
- **Sözlük** — her öğrenci ve rapor
- **Demet** — notlar: girildikten sonra değişmez
- **Kapsam** — ortalama() içindeki notlar yerel

KENDİNİ SINA

Çalıştırmadan tahmin et: ekranda ne görünür?

Soru 1 • Referans

```
a = [1, 2, 3]
b = a
a += [4]
print(b)
```

[1, 2, 3, 4]

a ve b aynı listeyi gösterir; += listeyi yerinde genişletir.

Soru 2 • Demet içinde liste

```
t = (1, [2, 3])
t[1].append(4)
print(t)
# Hata mı verir?
```

(1, [2, 3, 4])

Hata yok. Demet hangi nesneleri tuttuğunu değiştirmez; ama içindeki liste değişebilir.

DİKKAT

En sık yapılan 6 hata

`notlar[len(notlar)]`

Son indeks **len – 1**'dir → IndexError. Son eleman için **[-1]**.

`l = l.append(5)`

append **None** döndürür; liste kaybolur. Sadece **l.append(5)** yazın.

`t = (5)`

Bu bir int'tir. Tek elemanlı demet: **(5,)**

`d["yok"]`

Olmayan anahtar → KeyError. Emin değilseniz **d.get("yok")**.

`kopya = asil`

Kopya değil, ikinci isim. Gerçek kopya: **asil.copy()**

`list = [1, 2]`

Yerleşik **list** gölgelenir; sonra list() çalışmaz. Başka isim seçin.

⚠ Bonus: fonksiyon içinde global sayacı **sayac += 1** ile artırmak → UnboundLocalError.

ÖZET

Bugün öğrendiklerimiz, dört cümlede

[ ] Liste - List

Sıralı ve değiştirilebilir. İndeks 0'dan başlar; dilimleme yeni liste verir; **b = a** kopyalamaz.

( ) Demet - Tuple

Sıralı ama değiştirilemez. Virgül oluşturur; açılabilir; fonksiyonlar çoklu değeri demetle döndürür.

{ } Sözlük - Dictionary

Anahtar → değer. Anahtarlar benzersiz ve değişmez olmalı; olmayabilecek anahtar için **get()**.

LEGB Kapsam - Scope

İsim içeriden dışarıya aranır. İçeride atama yerel değişken yaratır; değiştirmek için **global** / **nonlocal**.

Doğru veri yapısını seçmek, kodun yarısını yazmaktır.

PRATİK ÖDEV · GELECEK HAFTA

Elini kirletme zamanı

1 · Kütüphane

Kitapları sözlük listesi olarak tut (ad, yazar, yıl). 2000'den sonra basılanları döndüren bir fonksiyon yaz.

2 · Harf sayacı

Bir cümledeki harflerin frekans sözlüğünü oluştur; en sık geçen 3 harfi yazdır.

3 · En yakın nokta

(x, y) demetlerinden oluşan bir listede orijine en yakın noktayı bul.

4 · Kapsam dedektifi

LEGB örneğindeki x atamalarını sırayla sil; her adımda çıktıyı tahmin et, sonra çalıştırıp doğrula.

HAFTA 2

Prosedürelden nesne yönelimliye geçiş

Bugünkü öğrenci sözlüğü, gelecek hafta bir **sınıfa** dönüşmeye başlıyor:

```
{"ad": "Erhan AKAGÜNDÜZ"}
↓
ogr = Ogrenci("Erhan AKAGÜNDÜZ")
```